Questions

1. What is the difference between `malloc()` and `calloc()`?

A) `malloc()` initializes memory, `calloc()` does not
B) `calloc()` initializes memory to zero, `malloc()` does not
C) `malloc()` uses more memory than `calloc()`
D) `calloc()` allocates memory in stack, `malloc()` in heap

Answer: B

2. What is the default return type of a function in C if not specified?

A) `int`
B) `void`
C) `float`
D) `double`

Answer: A

3. Which of the following is not a valid storage class in C?

A) `static`
B) `extern`
C) `mutable`
D) `register`

Answer: C

4. What will `sizeof(int*)` return on a 64-bit machine?

A) 2
B) 4
C) 8
D) 16

Answer: C

5. What is the primary purpose of the `volatile` keyword?

A) Prevents compiler optimizations on a variable
B) Allocates memory dynamically
C) Optimizes the execution of a loop
D) Increases memory efficiency

Answer: A

6. What does the `const` keyword do in C?

A) Allocates memory in heap
B) Makes a variable unmodifiable
C) Reserves space for dynamic allocation
D) Makes a variable global

Answer: B

7. Which of the following is true about function pointers?

A) They store addresses of functions
B) They must be used with global functions only
C) They can store only `void` functions
D) They do not exist in C

Answer: A

8. What is a segmentation fault in C?

A) Memory allocation error
B) Division by zero
C) Accessing memory not allocated to the program
D) Infinite loop error

Answer: C

9. Which of these is used to open a file in read mode in C?

A) `fopen(filename, "w")`
B) `fopen(filename, "a")`
C) `fopen(filename, "r")`
D) `fopen(filename, "rb")`

Answer: C

10. What is the output of `printf("%c", 'A' + 1);`?

A) A
B) B
C) 66
D) Compilation error

Answer: B

11. What will happen if `free()` is called on an already freed pointer?

A) Program terminates normally
B) Undefined behavior
C) Compiler warning
D) Memory leak

Answer: B

12. What does the `extern` keyword do?

A) Declares a function in another file
B) Allocates memory
C) Creates a global function
D) Converts function to inline

Answer: A

13. What is the output of `printf("%d", -1U > 1);`?

A) 0
B) 1
C) -1
D) Compilation error

Answer: B

14. What will be the output of `sizeof("Hello")`?

A) 5
B) 6
C) 7
D) Compiler error

Answer: B

15. Which of the following data types has the highest memory size?

A) `char`
B) `int`
C) `double`
D) `float`

Answer: C

---

2. Code Output Questions (20 Questions)

16. What will be the output?

c

```c
#include <stdio.h>
int main() {
    int a = 5;
    printf("%d %d %d", a++, a--, ++a);
    return 0;
}
```

A) 6 5 6
B) Undefined behavior
C) 5 6 5
D) 6 5 5

Answer: B

17. Find the output of the following code.

c

```c
#include <stdio.h>
int main() {
    char str[] = "Hello";
    char *p = str;
    printf("%c", ++*p);
    return 0;
}
```

A) H
B) I
C) Undefined behavior
D) Compiler error

Answer: B

18. What will be the output?

c

```c
#include <stdio.h>
int main() {
    int arr[] = {1, 2, 3, 4, 5};
    printf("%d", *(arr + 3));
    return 0;
}
```

A) 1
B) 3
C) 4
D) 5

Answer: C

(Continuing the pattern for 19–35 output questions.)

Code Output Questions (19-35)

19. What will be the output?

```c
#include <stdio.h>
int main() {
    int x = 10;
    int y = x++ + ++x;
    printf("%d", y);
    return 0;
}
```

A) 20
B) 21
C) Undefined behavior
D) 22
Answer: C

20. What will be the output?

```c
#include <stdio.h>
int main() {
    int a = 5, b = 2;
    printf("%d", a / b);
```

```
    return 0;
}
```

A) 2.5
B) 2
C) 3
D) Compilation error
Answer: B

21. What will be the output?

```
#include <stdio.h>
int main() {
    int arr[] = {10, 20, 30, 40};
    printf("%d", *(arr + 2));
    return 0;
}
```

A) 10
B) 20
C) 30
D) 40
Answer: C

22. What will be the output?

```
#include <stdio.h>
int main() {
    int i = 1;
    while (i < 5)
        printf("%d ", i++);
    return 0;
}
```

A) 1 2 3 4
B) 1 2 3 4 5
C) Infinite loop
D) Compilation error
Answer: A

23. What will be the output?

```
#include <stdio.h>
int main() {
    int x = 5;
```

```
    printf("%d", x << 1);
    return 0;
}
```

A) 5
B) 10
C) 2
D) 1
Answer: B

24. What will be the output?

```
#include <stdio.h>
int main() {
    int x = 5, y = 10;
    printf("%d", x > y ? x : y);
    return 0;
}
```

A) 5
B) 10
C) Compilation error
D) Undefined behavior
Answer: B

25. What will be the output?

```
#include <stdio.h>
int main() {
    int x = 0;
    if (x = 1)
        printf("True");
    else
        printf("False");
    return 0;
}
```

A) True
B) False
C) Compilation error
D) Undefined behavior
Answer: A

26. What will be the output?

```
#include <stdio.h>
int main() {
    char str[] = "C Programming";
    printf("%c", str[2]);
    return 0;
}
```

A) C
B) P
C) o
D) Compilation error
Answer: B

27. What will be the output?

```
#include <stdio.h>
int main() {
    int arr[] = {1, 2, 3, 4, 5};
    printf("%d", arr[5]);
    return 0;
}
```

A) 5
B) Undefined behavior
C) Compilation error
D) 0
Answer: B

28. What will be the output?

```
#include <stdio.h>
int main() {
    int i = 5;
    printf("%d", i++);
    return 0;
}
```

A) 4
B) 5
C) 6
D) Compilation error
Answer: B

29. What will be the output?

```
#include <stdio.h>
int main() {
    int x = 5, y = 10;
    int *p = &x, *q = &y;
    printf("%d", *p + *q);
    return 0;
}
```

A) 10
B) 15
C) 5
D) Compilation error
Answer: B

30. What will be the output?

```
#include <stdio.h>
#define SQUARE(x) x*x
int main() {
    printf("%d", SQUARE(5 + 1));
    return 0;
}
```

A) 36
B) 30
C) Compilation error
D) 11
Answer: B
Explanation: `SQUARE(5 + 1)` *expands to* `5 + 1 * 5 + 1`*, which is* `5 + 5 + 1 = 11`*.*

31. What will be the output?

```
#include <stdio.h>
int main() {
    int x = 5;
    printf("%d", x & 1);
    return 0;
}
```

A) 1
B) 0
C) 5
D) Compilation error
Answer: A

32. What will be the output?

```
#include <stdio.h>
int main() {
    int x = 5;
    printf("%d", x | 1);
    return 0;
}
```

A) 1
B) 4
C) 5
D) 7
Answer: C

33. What will be the output?

```
#include <stdio.h>
int main() {
    char ch = 'A' + 3;
    printf("%c", ch);
    return 0;
}
```

A) A
B) B
C) D
D) Compilation error
Answer: C

34. What will be the output?

```
#include <stdio.h>
int main() {
    int x = 5;
    int y = x++;
    printf("%d %d", x, y);
    return 0;
}
```

A) 5 5
B) 6 5
C) 6 6
D) Compilation error
Answer: B

35. What will be the output?

```
#include <stdio.h>
int main() {
    char str[] = "Hello";
    printf("%c", *(str + 4));
    return 0;
}
```

A) H
B) e
C) o
D) Compilation error
Answer: C

3. Problem-Solving & Debugging (15 Questions)

36. Which of the following is the correct way to swap two numbers without using a third variable?

A) `a = a + b; b = a - b; a = a - b;`
B) `a = a ^ b; b = a ^ b; a = a ^ b;`
C) `a = b - a; b = b - a; a = b + a;`
D) Both A & B

Answer: D

37. What is the time complexity of binary search?

A) O(n)
B) O(log n)
C) O(n log n)
D) O(1)

Answer: B

38. Which sorting algorithm has the worst-case time complexity of $O(n^2)$?

A) Quick Sort
B) Merge Sort

C) Bubble Sort
D) Heap Sort

Answer: C

39. What is the output of `printf("%d", 5 & 3);`?

A) 2
B) 3
C) 5
D) 8

Answer: A

40. What will `printf("%d", sizeof(NULL));` return?

A) 1
B) 2
C) 4
D) Compilation error

Answer: D

41. Which of the following is a valid keyword in C?

A) `function`
B) `virtual`
C) `typedef`
D) `class`

Answer: C

42. What is the primary difference between `while` and `do-while` loops?

A) `while` executes at least once, `do-while` may not execute
B) `do-while` executes at least once, `while` may not execute
C) Both are identical
D) `do-while` is faster than `while`

Answer: B

43. What does `printf("%d", 5 / 2);` output?

A) 2.5
B) 2
C) 3
D) Compilation error

Answer: B

44. What does `break` do inside a loop?

A) Terminates the loop execution
B) Skips the next iteration
C) Restarts the loop from the beginning
D) Causes a syntax error

Answer: A

45. What is the return type of the `main()` function in C?

A) `void`
B) `char`
C) `int`
D) `float`

Answer: C

46. What is the output of `sizeof(float)` on a 32-bit system?

A) 2
B) 4
C) 8
D) 16

Answer: B

47. What is a NULL pointer in C?

A) A pointer that points to an unallocated memory location
B) A pointer that stores 0 as its address
C) A pointer that points to garbage value
D) A pointer that stores random values

Answer: B

48. What is the default value of an uninitialized global variable in C?

A) 0
B) Garbage value
C) NULL
D) Compiler error

Answer: A

49. Which of the following operators has the highest precedence in C?

A) `+`
B) `*`
C) `=`
D) `&&`

Answer: B

50. Which function is used to deallocate memory in C?

A) `delete()`
B) `free()`
C) `release()`
D) `clear()`

Answer: B

51. What is the format specifier for printing an `unsigned int`?

A) `%d`
B) `%u`
C) `%x`
D) `%f`

Answer: B

52. Which of the following is not a valid way to declare an integer variable in C?

A) `int x = 10;`
B) `int x(10);`
C) `int x; x = 10;`
D) `int x = {10};`

Answer: B

53. What is the output of `printf("%c", 'A' + 2);`?

A) A
B) B
C) C
D) Compilation error

Answer: C

54. Which of these is used to read a line of text from the user in C?

A) `scanf("%s", str);`
B) `gets(str);`
C) `fgets(str, sizeof(str), stdin);`
D) Both B and C

Answer: D

55. How do you declare a constant in C?

A) `int const x = 10;`
B) `#define x 10`
C) Both A & B
D) None of the above

Answer: C

---

2. Code Output Questions (20 Questions, 56–75)

56. Find the output:

c

```c
#include <stdio.h>

int main() {

    int a = 5;
```

```
    printf("%d", a++);

    return 0;

}
```

A) 4
B) 5
C) 6
D) Compilation error

Answer: B

57. What is the output?

c

```
#include <stdio.h>

int main() {

    char str[] = "Hello";

    printf("%c", *(str + 2));

    return 0;

}
```

A) H
B) e
C) l
D) o

Answer: C

58. Predict the output:

c

```c
#include <stdio.h>

int main() {

    int x = 2;

    x = x << 2;

    printf("%d", x);

    return 0;

}
```

A) 2
B) 4
C) 8
D) 16

Answer: C

59. What is the output of `printf("%d", sizeof(NULL));`?

A) 1
B) 4
C) 8
D) Compilation error

---

60. What does `int (*ptr)[10];` mean?

A) `ptr` is an array of 10 integer pointers
B) `ptr` is a pointer to an array of 10 integers
C) `ptr` is a function returning an array of 10 integers
D) `ptr` is a constant integer pointer

---

61. What is the output of `5 / 2 * 2.0`?

A) 5.0
B) 2.5
C) 4.0
D) 5

---

62. What is the output of `int x = 2; printf("%d", x *= 3 + 2);`?

A) 6
B) 8
C) 10
D) 12

---

63. What is the output of `printf("%d", 'A' - 'B');`?

A) 1
B) -1
C) 0
D) 2

---

64. What will `printf("%d", -1U);` print?

A) -1
B) Large positive integer
C) 0
D) Compilation error

---

65. What is the output of `printf("%d", (int)3.14);`?

A) 3.14
B) 3
C) 4
D) Compilation error

---

66. What does `int *const p;` mean?

A) `p` is a constant integer pointer
B) `p` is a pointer to a constant integer
C) `p` is a constant pointer to a constant integer
D) `p` is a constant pointer to an integer

---

67. What does `void (*f)();` mean?

A) `f` is a function that takes `void` and returns `void`
B) `f` is a pointer to an integer
C) `f` is a pointer to a function returning `void`
D) `f` is a function pointer returning an integer

---

68. What is the output of `printf("%d", 2 | 3);`?

A) 2
B) 3
C) 1
D) 0

---

69. What is the result of `sizeof(1.5f);`?

A) 8
B) 4
C) 2
D) 1

---

70. What is the output of `sizeof("A");`?

A) 1
B) 2
C) 4
D) 8

---

71. What is the result of `printf("%d", 10 && 0);`?

A) 10
B) 0
C) 1
D) Compilation error

---

72. What does `int arr[5] = {1,2,3};` initialize `arr[4]` to?

A) 0
B) 3
C) Garbage value
D) Compilation error

---

73. What is the output of `printf("%d", (5,10,15));`?

A) 5
B) 10
C) 15
D) Compilation error

---

74. What is the output of `int x = 5, y = 10; printf("%d", x < y ? x++ : y++);`?

A) 6
B) 5
C) 10
D) 11

---

75. What does `static int x;` initialize `x` to?

A) 0
B) 1
C) Garbage value
D) Compilation error

---

3. Problem-Solving & Debugging (15 Questions, 76–90)

76. Which sorting algorithm has the best average-case performance?

A) Bubble Sort
B) Merge Sort
C) Selection Sort
D) Insertion Sort

Answer: B

77. What is the time complexity of an optimal binary search algorithm?

A) O(n)
B) O(log n)
C) O(n log n)
D) O(1)

Answer: B

78. What is the space complexity of recursive Fibonacci calculation?

A) O(1)
B) O(n)
C) O(log n)
D) O(n²)

Answer: B

79. Identify the error in the following code:

c

```
#include <stdio.h>

int main() {
```

```
    int arr[5] = {1, 2, 3, 4, 5};

    printf("%d", arr[5]);

    return 0;

}
```

A) Array index out of bounds
B) Syntax error
C) Compilation error
D) No error

Answer: A

80. How can you efficiently check if a number is a power of 2?

A) `(n & (n - 1)) == 0`
B) `(n | (n - 1)) == 0`
C) `(n >> 1) == 0`
D) `n % 2 == 0`

Answer: A

81. How do you dynamically allocate an array in C?

A) `int *arr = malloc(n * sizeof(int));`
B) `int arr[n];`
C) `int arr = new int[n];`
D) `int arr[] = {1, 2, 3};`

Answer: A

82. What is the maximum value of an `unsigned char`?

A) 127
B) 255
C) 65535
D) 1024

Answer: B

83. What will be the output of the following code?

c

```c
#include <stdio.h>

int main() {

    int x = 10;

    int *ptr = &x;

    *ptr = *ptr + 5;

    printf("%d", x);

    return 0;

}
```

A) 10
B) 15
C) Address of x
D) Compilation error

Answer: B

---

84. What is the output of this program?

c

```c
#include <stdio.h>

int main() {

    int a = 5;

    if (a = 0)
```

```
        printf("Zero");
    else
        printf("Not Zero");
    return 0;
}
```

A) Zero
B) Not Zero
C) Compilation error
D) Undefined behavior

Answer: B

Explanation: `a = 0` is an assignment, not a comparison, so `if (0)` evaluates to `false`, and "Not Zero" is printed.

---

85. Which of the following statements about pointers in C is true?

A) A `NULL` pointer points to an invalid memory location.
B) `&` is used to access the value at the memory address stored in a pointer.
C) A pointer can store any data type except `void`.
D) `malloc()` always returns an integer pointer.

Answer: A

---

86. What will be the output of the following program?

c

```
#include <stdio.h>

int main() {
```

```
    char str[] = "DAIICT";

    printf("%c", *(str + 3));

    return 0;

}
```

A) A
B) I
C) C
D) T

Answer: B

Explanation: `*(str + 3)` accesses the 4th character of `"DAIICT"`, which is `'I'`.

---

87. What is the output of the following code?

c

```
#include <stdio.h>

#define SQUARE(x) x*x

int main() {

    int a = 4;

    printf("%d", SQUARE(a + 1));

    return 0;

}
```

A) 25
B) 9

C) 20
D) Compilation error

Answer: C

Explanation: `SQUARE(a + 1)` expands to `a + 1 * a + 1`, which evaluates as `4 + 1 * 4 + 1 = 4 + 4 + 1 = 9`. To fix this, use parentheses: `#define SQUARE(x) ((x) * (x))`.

---

88. Which of the following statements is true regarding `malloc()` and `calloc()`?

A) `malloc()` initializes allocated memory to zero, whereas `calloc()` does not.
B) `calloc()` initializes allocated memory to zero, whereas `malloc()` does not.
C) Both `malloc()` and `calloc()` initialize memory to zero.
D) Neither `malloc()` nor `calloc()` initialize memory to zero.

Answer: B

---

89. Which of the following expressions correctly determines if an integer `x` is even?

A) `x % 2 == 0`
B) `x & 1 == 0`
C) `x / 2 == 0`
D) Both A and B

Answer: D

Explanation: `x % 2 == 0` checks if `x` is divisible by 2, and `x & 1 == 0` checks if the least significant bit is 0, both indicating an even number.

---

90. What will be the output of this program?

c

```
#include <stdio.h>
```

```
void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}
int main() {
    int x = 5, y = 10;
    swap(&x, &y);
    printf("%d %d", x, y);
    return 0;
}
```

A) 5 10
B) 10 5
C) Compilation error
D) Undefined behavior

Answer: B

Solutions

1. The difference between `malloc()` and `calloc()`

Answer:
✅ **B) `calloc()` initializes memory to zero, `malloc()` does not**
📌 `malloc(size)` allocates memory but does not initialize it, whereas `calloc(n, size)` allocates and sets all bytes to zero.

---

2. Default return type of a function in C

Answer:
✅ **A) int**
📌 If no return type is specified, `int` is assumed.

---

3. Invalid storage class in C

Answer:
✅ **C) mutable**
📌 `mutable` is a C++ keyword, not a valid C storage class.

---

4. `sizeof(int*)` on a 64-bit machine

Answer:
✅ **C) 8**
📌 Pointer size depends on the system architecture. In a 64-bit system, it's 8 bytes.

---

5. Primary purpose of `volatile` keyword

Answer:
✅ **A) Prevents compiler optimizations on a variable**
📌 Used for hardware registers, global variables in multi-threaded applications.

---

6. Effect of `const` keyword

Answer:
✅ **B) Makes a variable unmodifiable**
📌 Declaring `const int x = 5;` means `x` cannot be changed.

---

7. True about function pointers

Answer:
✅ **A) They store addresses of functions**
📌 Function pointers allow calling a function indirectly.

---

8. Segmentation fault in C

Answer:
✅ **C) Accessing memory not allocated to the program**
📌 This occurs when accessing an invalid pointer or memory outside allocated space.

---

9. Opening a file in read mode

Answer:
✅ **C) `fopen(filename, "r")`**
📌 `"r"` mode opens a file for reading.

---

10. Output of `printf("%c", 'A' + 1);`

Answer:
✅ **B) B**
📌 `'A' + 1` → `'B'` as ASCII of `'A'` is 65, and `'B'` is 66.

---

11. Calling `free()` on an already freed pointer

Answer:
✅ **B) Undefined behavior**
📌 Double free may cause a crash or unpredictable results.

---

12. Purpose of `extern` keyword

Answer:
✅ **A) Declares a function in another file**
📌 Used for global variable declarations across multiple files.

---

13. Output of `printf("%d", -1U > 1);`

Answer:
✅ **B) 1**
📌 `-1U` is an unsigned integer, which becomes a large positive number.

---

14. Output of `sizeof("Hello")`

Answer:
✅ **B) 6**
📌 `"Hello"` includes a null terminator `\0`, making its size 6.

---

15. Data type with the highest memory size

Answer:
✅ **C) double**
📌 Typically, `char = 1 byte`, `int = 4 bytes`, `float = 4 bytes`, `double = 8 bytes`.

Code Output Questions

(16-35)

16. Output of

c

```
int a = 5;
printf("%d %d %d", a++, a--, ++a);
```

✅ **B) Undefined behavior**
📌 Modifying `a` multiple times without sequence points results in UB.

---

17. Output of
c

```
char str[] = "Hello";
char *p = str;
printf("%c", ++*p);
```

✅ **B) I**
📌 `++*p` increments `'H'` (ASCII 72) to `'I'`.

---

18. Output of
c

```
int arr[] = {1, 2, 3, 4, 5};
printf("%d", *(arr + 3));
```

✅ **C) 4**
📌 `arr + 3` points to `arr[3]`, which is `4`.

---

19. Output of
c

```
int x = 10;
int y = x++ + ++x;
printf("%d", y);
```

✅ **C) Undefined behavior**
📌 Order of evaluation is unspecified.

---

20. Output of

c

```
int a = 5, b = 2;
printf("%d", a / b);
```

✅ **B) 2**
📌 Integer division truncates decimal places.

---

21. Output of
c

```
int arr[] = {10, 20, 30, 40};
printf("%d", *(arr + 2));
```

✅ **C) 30**

---

22. Output of
c

```
int i = 1;
while (i < 5)
    printf("%d ", i++);
```

✅ **A) 1 2 3 4**

---

23. Output of
c

```
int x = 5;
printf("%d", x << 1);
```

✅ **B) 10**
📌 Left shift (<<) doubles the value.

---

24. Output of
c

```
int x = 5, y = 10;
printf("%d", x > y ? x : y);
```

✅ **B) 10**
📌 Ternary operator picks `y` since `x > y` is false.

---

25. Output of
c

```
int x = 0;
if (x = 1)
    printf("True");
else
    printf("False");
```

✅ **A) True**
📌 `if (x = 1)` assigns `1` to `x`, making it always true.

---

26. Output of
c

```
char str[] = "C Programming";
printf("%c", str[2]);
```

✅ **C) o**

---

27. Output of
c

```
int arr[] = {1, 2, 3, 4, 5};
```

```c
printf("%d", arr[5]);
```

✅ **B) Undefined behavior**
📌 Out-of-bounds access.

---

28. Output of
c

```c
int i = 5;
printf("%d", i++);
```

✅ **B) 5**
📌 Post-increment prints before incrementing.

---

29. Output of
c

```c
int x = 5, y = 10;
int *p = &x, *q = &y;
printf("%d", *p + *q);
```

✅ **B) 15**

---

30. Output of
c

```c
#define SQUARE(x) x*x
printf("%d", SQUARE(5 + 1));
```

✅ **B) 30**
📌 Expands to `5 + 1 * 5 + 1 = 30`.

Problem-Solving & Debugging

(36-50)

36. Swapping two numbers without a third variable

✅ **D) Both A & B**
📌 Arithmetic and bitwise XOR methods work.

---

37. Time complexity of binary search

✅ **B) O(log n)**

---

38. Sorting algorithm with worst-case $O(n^2)$

✅ **C) Bubble Sort**

---

39. Output of
c

```c
printf("%d", 5 & 3);
```

✅ **A) 1**

---

40. Output of
c

```c
printf("%d", sizeof(NULL));
```

✅ **D) Compilation error**

---

41. Which of the following is a valid keyword in C?
✅ **C) typedef**

`typedef` is a keyword used to create type aliases. Other options (`function`, `virtual`, `class`) are not valid in C.

---

42. What is the primary difference between `while` and `do-while` loops?
✅ **B) do-while executes at least once, while may not execute**
A `do-while` loop executes the body before checking the condition, while `while` checks the condition first.

---

43. Output of

c

```
printf("%d", 5 / 2);
```

✅ **B) 2**
Integer division truncates the decimal part, so `5 / 2` results in `2`, not `2.5`.

---

44. What does `break` do inside a loop?
✅ **A) Terminates the loop execution**
The `break` statement immediately exits the loop, regardless of the loop condition.

---

45. What is the return type of the `main()` function in C?
✅ **C) int**
In standard C, `main()` should return an `int` to indicate the program's execution status.

---

46. What is the output of `sizeof(float)` on a 32-bit system?
✅ **B) 4**
A `float` typically takes 4 bytes (32 bits) on a 32-bit system.

---

47. What is a NULL pointer in C?
✅ **B) A pointer that stores 0 as its address**
A NULL pointer points to address `0`, indicating it does not reference any valid memory.

---

48. What is the default value of an uninitialized global variable in C?
✅ **A) 0**
Global variables are automatically initialized to `0` by the compiler.

---

49. Which of the following operators has the highest precedence in C?
✅ **B) ***
Multiplication (`*`) has higher precedence than addition (`+`), assignment (`=`), and logical AND (`&&`).

---

50. Which function is used to deallocate memory in C?
✅ **B) free()**
The `free()` function releases dynamically allocated memory back to the system.

---

51. What is the format specifier for printing an `unsigned int`?
✅ **B) %u**
The `%u` format specifier prints unsigned integers, while `%d` is used for signed integers.

---

52. Which of the following is not a valid way to declare an integer variable in C?
✅ **B) int x(10);**
C does not support constructor-style initialization (`int x(10);`), unlike C++.

---

53. Output of

c

```
printf("%c", 'A' + 2);
```

✅ **C) C**
ASCII value of `'A'` is `65`, so `'A' + 2` results in `67`, which corresponds to `'C'`.

---

54. Which of these is used to read a line of text from the user in C?
✅ **D) Both B and C**
Both `gets()` and `fgets()` can read a string, but `gets()` is unsafe and deprecated.

---

55. How do you declare a constant in C?
✅ **C) Both A & B**
Constants can be declared using `#define` or `const int x = 10;`.

---

56. Output of

c

```
int a = 5;
printf("%d", a++);
```

✅ **B) 5**
Post-increment returns `a` before incrementing it, so it prints `5`.

---

57. Output of

c

```
char str[] = "Hello";
printf("%c", *(str + 2));
```

✅ **C) l**
`*(str + 2)` accesses the third character in `"Hello"`, which is `'l'`.

---

58. Output of

c

```
int x = 2;
x = x << 2;
printf("%d", x);
```

✅ **C) 8**
Left shift by `2` (`x << 2`) is equivalent to multiplying by $2^2 = 4$, so `2 * 4 = 8`.

---

59. Output of

c

```
printf("%d", sizeof(NULL));
```

✅ **C) 8**
On a 64-bit system, `sizeof(NULL)` equals `sizeof(void*)`, which is `8` bytes.

---

60. What does `int (*ptr)[10];` mean?
✅ **B) ptr is a pointer to an array of 10 integers**
The parentheses indicate that `ptr` is a pointer to an array of `10` integers, not an array of pointers.

---

61. Output of

c

```
printf("%f", 5 / 2 * 2.0);
```

✅ **C) 4.0**
Integer division `5 / 2` results in `2`, then `2 * 2.0` gives `4.0`.

---

62. Output of

c

```
int x = 2;
printf("%d", x *= 3 + 2);
```

✅ **D) 12**
`x *= 3 + 2` is evaluated as `x = x * (3 + 2) = 2 * 6 = 12`.

---

63. Output of

c

```
printf("%d", 'A' - 'B');
```

✅ **B) -1**
ASCII value of `'A'` is `65`, `'B'` is `66`, so `65 - 66 = -1`.

---

64. Output of

c

```
printf("%d", -1U);
```

✅ **B) Large positive integer**
`-1U` is interpreted as an unsigned integer, wrapping around to the maximum positive value.

---

65. Output of

c

```
printf("%d", (int)3.14);
```

✅ **B) 3**
Explicit casting `(int)3.14` truncates the decimal part, giving `3`.

---

66. What does `int *const p;` mean?
✅ **D) p is a constant pointer to an integer**

The `const` after `*` means the pointer itself is constant, but the integer it points to can be changed.

---

67. What does `void (*f)();` mean?
✅ **C) f is a pointer to a function returning void**
The `(*f)()` notation means `f` is a pointer to a function, and `void` indicates it returns nothing.

---

68. Output of `printf("%d", 2 | 3);`
✅ **B) 3**
Bitwise OR: `2 (10) | 3 (11) = 3 (11)`, so the output is `3`.

---

69. What is the result of `sizeof(1.5f);`?
✅ **B) 4**
A `float` typically occupies `4` bytes in memory.

---

70. What is the output of `sizeof("A");`?
✅ **B) 2**
A string `"A"` includes the null terminator (`'\0'`), so it takes `2` bytes.

---

71. What is the result of `printf("%d", 10 && 0);`?
✅ **B) 0**
Logical AND (`&&`) returns `1` if both operands are nonzero, else `0`. Since `0` is present, the result is `0`.

---

72. What does `int arr[5] = {1,2,3};` initialize `arr[4]` to?
✅ **A) 0**
Uninitialized elements in an array declaration default to `0`.

---

73. Output of `printf("%d", (5,10,15));`
✅ **C) 15**
The comma operator evaluates all expressions and returns the last one, which is `15`.

---

74. Output of

c

```
int x = 5, y = 10;
printf("%d", x < y ? x++ : y++);
```

✅ **B) 5**
Since `x < y` is true, `x` is returned before incrementing (`x++`), so `5` is printed.

---

75. What does `static int x;` initialize `x` to?
✅ **A) 0**
Static variables are initialized to `0` by default.

Problem-Solving & Debugging

76. Which sorting algorithm has the best average-case performance?
✅ **B) Merge Sort**
Merge Sort runs in `O(n log n)` on average, which is better than Bubble or Selection Sort (`O(n²)`).

---

77. What is the time complexity of an optimal binary search algorithm?
✅ **B) O(log n)**
Binary search repeatedly divides the search space in half, leading to logarithmic time complexity.

---

78. What is the space complexity of recursive Fibonacci calculation?
✅ **B) O(n)**
Recursive Fibonacci uses `O(n)` space due to the recursion stack.

---

79. Identify the error in this code:

c

```c
int arr[5] = {1, 2, 3, 4, 5};
printf("%d", arr[5]);
```

✅ **A) Array index out of bounds**
Arrays in C are zero-indexed, so `arr[5]` is out of bounds for a 5-element array.

---

80. How can you efficiently check if a number is a power of 2?
✅ **A)** **`(n & (n - 1)) == 0`**
This expression is true for powers of 2 because powers of 2 have only one `1` bit in binary.

---

81. How do you dynamically allocate an array in C?
✅ **A)** **`int *arr = malloc(n * sizeof(int));`**
`malloc` dynamically allocates memory, whereas `int arr[n];` is only valid in C99+.

---

82. What is the maximum value of an `unsigned char`?
✅ **B) 255**
An `unsigned char` uses `8` bits, so its range is `0-255`.

---

83. Output of this code:

c

```c
int x = 10;
int *ptr = &x;
*ptr = *ptr + 5;
printf("%d", x);
```

✅ **B) 15**
Dereferencing `ptr` updates `x`, so `10 + 5 = 15`.

---

84. Output of this program:

c

```
int a = 5;
if (a = 0)
    printf("Zero");
else
    printf("Not Zero");
```

✅ **B) Not Zero**
`a = 0` assigns `0` to `a`, but since `if (0)` is false, `"Not Zero"` is printed.

---

85. Which of these is true about pointers?
✅ **A) A NULL pointer points to an invalid memory location.**
A `NULL` pointer is a special pointer that does not point to any valid memory.

---

86. Output of this program:

c

```
char str[] = "DAIICT";
printf("%c", *(str + 3));
```

✅ **B) I**
`*(str + 3)` refers to the fourth character, which is `'I'`.

---

87. Output of this program:

c

```
#define SQUARE(x) x*x
```

```
int a = 4;
printf("%d", SQUARE(a + 1));
```

✅ **C) 20**
`SQUARE(a + 1)` expands to `a + 1 * a + 1`, which evaluates incorrectly as `4 + 1 * 4 + 1 = 20`.

---

88. Which is true regarding `malloc()` and `calloc()`?
✅ **B) `calloc()` initializes allocated memory to zero, whereas `malloc()` does not.**
`malloc()` does not initialize memory, while `calloc()` initializes all bytes to `0`.

---

89. Which expressions correctly check if an integer `x` is even?
✅ **D) Both A and B**
`x % 2 == 0` checks divisibility, while `x & 1 == 0` checks the least significant bit.

---

90. Output of this program:

c

```
void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}
int main() {
    int x = 5, y = 10;
    swap(&x, &y);
    printf("%d %d", x, y);
}
```

✅ **B) 10 5**
Since `swap()` uses pointers, `x` and `y` get swapped correctly.
